

AI Lab -- Complete Exam Cheat Sheet

All Labs · All Scenarios · Copy-Paste Ready Code

CSP | Minimax | Alpha-Beta | Bayesian Networks | Markov | EDA | Supervised | Unsupervised

LAB 7 -- Constraint Satisfaction Problems (CSP)

Concept

A CSP has three parts: **Variables** (X), **Domains** (D), **Constraints** (C). Goal: assign each variable a value from its domain so ALL constraints are satisfied.

! Always check: consistent = no violated constraints. Complete = every variable assigned.

Basic Template

```
from ortools.sat.python import cp_model
model = cp_model.CpModel()

x = model.NewIntVar(1, 5, 'x')      # x can be 1..5
y = model.NewIntVar(1, 5, 'y')
presentations = [x, y]

model.AddAllDifferent(presentations) # no two same value
model.Add(x != 1)                    # simple constraint
model.Add(x < y)                      # ordering

# Absolute difference (consecutive check)
diff = model.NewIntVar(-4, 4, 'diff')
abs_d = model.NewIntVar(0, 4, 'abs_d')
model.Add(diff == x - y)
model.AddAbsEquality(abs_d, diff)
model.Add(abs_d != 1)                # not consecutive

# Boolean OR (slot reserved for x OR y)
bx = model.NewBoolVar('bx'); by = model.NewBoolVar('by')
model.Add(x == 3).OnlyEnforceIf(bx)
model.Add(x != 3).OnlyEnforceIf(bx.Not())
model.Add(y == 3).OnlyEnforceIf(by)
model.Add(y != 3).OnlyEnforceIf(by.Not())
model.AddBoolOr([bx, by])

solver = cp_model.CpSolver()
status = solver.Solve(model)
if status in (cp_model.FEASIBLE, cp_model.OPTIMAL):
    print('x =', solver.Value(x), ' y =', solver.Value(y))
else:
    print('No solution.')
```

Past Paper Q1 -- Conference Schedule

5 presentations (T1-T5), 5 time slots. Constraints: all different · T1&T3; not consecutive · T2 before T4 · T5 not in slots 1 or 2 · T3 after T1 · T4 not slot 5 · slot 3 for T1 or T2.

```
from ortools.sat.python import cp_model

model = cp_model.CpModel()
t1=model.NewIntVar(1,5, 'T1'); t2=model.NewIntVar(1,5, 'T2')
t3=model.NewIntVar(1,5, 'T3'); t4=model.NewIntVar(1,5, 'T4')
t5=model.NewIntVar(1,5, 'T5')
presentations = [t1,t2,t3,t4,t5]

model.AddAllDifferent(presentations)

diff=model.NewIntVar(-4,4, 'd'); abs_d=model.NewIntVar(0,4, 'ad')
model.Add(diff==t1-t3); model.AddAbsEquality(abs_d,diff)
model.Add(abs_d!=1)                # T1 & T3 not consecutive

model.Add(t2<t4)                    # T2 before T4
model.Add(t5!=1); model.Add(t5!=2) # T5 not morning
model.Add(t3>t1)                    # T3 after T1
model.Add(t4!=5)                    # T4 not last slot
```

```

b1=model.NewBoolVar('b1'); b2=model.NewBoolVar('b2')
model.Add(t1==3).OnlyEnforceIf(b1); model.Add(t1!=3).OnlyEnforceIf(b1.Not())
model.Add(t2==3).OnlyEnforceIf(b2); model.Add(t2!=3).OnlyEnforceIf(b2.Not())
model.AddBoolOr([b1,b2]) # slot 3 for T1 or T2

solver=cp_model.CpSolver()
status=solver.Solve(model)
slot_time={1:'9-10AM',2:'10-11AM',3:'11-12PM',4:'2-3PM',5:'3-4PM'}
if status in (cp_model.OPTIMAL,cp_model.FEASIBLE):
    for name,var in [('ML',t1),('Robotics',t2),('NLP',t3),('CV',t4),('RL',t5)]:
        s=solver.Value(var)
        print(f'{name:12s}-> Slot {s} ({slot_time[s]})')

```

Scenario -- Seating (6 students, 6 seats)

```

from ortools.sat.python import cp_model
model=cp_model.CpModel()
students=['Ali', 'Sara', 'Ahmed', 'Zoya', 'Bilal', 'Hina']
seats={s:model.new_int_var(1,6,s) for s in students}
model.add_all_different(seats.values())

# Sara next to Zoya (|Sara-Zoya|==1)
d=model.new_int_var(-5,5,'d'); a=model.new_int_var(0,5,'a')
model.add(d==seats['Sara']-seats['Zoya'])
model.add_abs_equality(a,d); model.add(a==1)

# Ali NOT next to Ahmed (|Ali-Ahmed|!=1)
d2=model.new_int_var(-5,5,'d2'); a2=model.new_int_var(0,5,'a2')
model.add(d2==seats['Ali']-seats['Ahmed'])
model.add_abs_equality(a2,d2); model.add(a2!=1)

model.add_allowed_assignments([seats['Bilal']],[(2,),(4,),(6,)])
model.add(seats['Hina']!=1)

solver=cp_model.CpSolver(); status=solver.solve(model)
if status in (cp_model.FEASIBLE,cp_model.OPTIMAL):
    for s in students: print(f' {s}: Seat {solver.value(seats[s])}')

```

Scenario -- Delivery Slots & Exam Schedule

```

# DELIVERY SLOTS
from ortools.sat.python import cp_model
model=cp_model.CpModel()
D={d:model.new_int_var(1,3,d) for d in ['D1','D2','D3','D4']}
model.add(D['D1']!=D['D2']); model.add(D['D3']<D['D4'])
model.add(D['D2']!=1)
solver=cp_model.CpSolver(); solver.solve(model)

# EXAM SCHEDULE
subjects=['Math','Physics','AI','OS','DB']
exam={s:model.new_int_var(1,6,s) for s in subjects}
model.add(exam['Math']!=exam['Physics']); model.add(exam['AI']!=exam['OS'])
model.add(exam['Math']<exam['DB'])
d=model.new_int_var(-6,6,'d'); a=model.new_int_var(0,6,'a')
model.add(d==exam['AI']-exam['OS']); model.add_abs_equality(a,d)
model.add(a>=2) # AI and OS gap >= 2 slots

```

LAB 8 -- Minimax & Alpha-Beta Pruning

Concepts

Minimax: MAX wants highest score, MIN wants lowest. Values bubble UP from leaves. At MAX level: pick max child. At MIN level: pick min child.

Alpha-Beta: Same result as Minimax, but FASTER -- prunes useless branches. alpha = best MAX guarantee (starts -inf). beta = best MIN guarantee (starts +inf). PRUNE when beta <= alpha.

! Alpha-Beta gives IDENTICAL answer to Minimax, just skips branches that can't change result.

Minimax -- Nested List Tree

```
import math

def minimax(tree, is_max):
    if isinstance(tree, int): return tree    # leaf node
    if is_max:
        best = float('-inf')
        for child in tree:
            val = minimax(child, False)
            best = max(best, val)
        return best
    else:
        best = float('inf')
        for child in tree:
            val = minimax(child, True)
            best = min(best, val)
        return best

tree = [[ [7,3],[6,5] ], [ [9,1],[5,11] ]]
print('Optimal =', minimax(tree, True))    # MAX starts
```

Alpha-Beta Pruning

```
import math

def alphabeta(tree, alpha, beta, is_max):
    if isinstance(tree, int): return tree
    if is_max:
        best = float('-inf')
        for child in tree:
            val = alphabeta(child, alpha, beta, False)
            best = max(best, val)
            alpha = max(alpha, best)
            if alpha >= beta: break    # beta cutoff - PRUNE!
        return best
    else:
        best = float('inf')
        for child in tree:
            val = alphabeta(child, alpha, beta, True)
            best = min(best, val)
            beta = min(beta, best)
            if alpha >= beta: break    # alpha cutoff - PRUNE!
        return best

tree = [[ [7,3],[6,5] ], [ [9,1],[5,11] ]]
result = alphabeta(tree, float('-inf'), float('inf'), True)
print('Optimal =', result)
```

Past Paper Q2 -- Strategy Cards (dict-based tree)

```
import math

TREE = {
    'ROOT': ['A', 'B', 'C', 'D'],
    'A': [7,3], 'B': [6,5], 'C': [9,1], 'D': [5,11],
```

```

}

def minimax(node, maximizing):
    if isinstance(node, int): return node, None
    children = TREE.get(node, [])
    best_move = None
    if maximizing:
        best = -math.inf
        for child in children:
            score, _ = minimax(child, False)
            if score > best: best, best_move = score, child
    else:
        best = math.inf
        for child in children:
            score, _ = minimax(child, True)
            if score < best: best, best_move = score, child
    return best, best_move

# (b) MIN picks for each card
for card in ['A', 'B', 'C', 'D']:
    print(f'Card {card}: MIN picks {min(TREE[card])}')

# (c) MAX best card
score, move = minimax('ROOT', maximizing=True)
print(f'MAX plays Card {move} -> Guaranteed payoff: {score}')
# Card A:3 Card B:5 Card C:1 Card D:5 => MAX picks B or D (payoff=5)

```

Scenario -- Coins Game (Alpha-Beta)

```

import math
coins = [3,9,1,2,7,5]

def ab_coins(coins, l, r, alpha, beta, maximizing):
    if l > r: return 0
    if maximizing:
        best=-math.inf
        s1=coins[l]+ab_coins(coins, l+1, r, alpha, beta, False)
        best=max(best,s1); alpha=max(alpha,best)
        if beta<=alpha: return best
        s2=coins[r]+ab_coins(coins, l, r-1, alpha, beta, False)
        return max(best,s2)
    else:
        best=math.inf
        s1=ab_coins(coins, l+1, r, alpha, beta, True)
        best=min(best,s1); beta=min(beta,best)
        if beta<=alpha: return best
        return min(best,ab_coins(coins, l, r-1, alpha, beta, True))

l,r=0,len(coins)-1; ms=0; mins=0; go_max=True
while l<=r:
    if go_max:
        pl=coins[l]+ab_coins(coins, l+1, r, -math.inf, math.inf, False)
        pr=coins[r]+ab_coins(coins, l, r-1, -math.inf, math.inf, False)
        if pl>=pr: p=coins[l]; l+=1
        else: p=coins[r]; r-=1
        ms+=p; print(f'Max picks {p}')
    else:
        if coins[l]<=coins[r]: p=coins[l]; l+=1
        else: p=coins[r]; r-=1
        mins+=p; print(f'Min picks {p}')
    go_max=not go_max
print(f'Max:{ms} Min:{mins}')

```

LAB 9 -- Bayesian Networks & Markov Models

BN Concept

DAG + CPTs. Nodes = variables. Edges = parent to child (arrows show dependency). CPT = probability table for each node given parent states. Root nodes have no parents -- just one column.

! CRITICAL: Each COLUMN in every CPT must sum exactly to 1.0!

BN Template (2-parent node has 4 columns)

```
from pgmpy.models import DiscreteBayesianNetwork
from pgmpy.factors.discrete import TabularCPD
from pgmpy.inference import VariableElimination

model = DiscreteBayesianNetwork([
    ('P1', 'Child'), ('P2', 'Child'), ('Child', 'Output')
])

# Root node -- no parents, just one column
cpd_p1 = TabularCPD('P1', 2, [[0.7],[0.3]]) # 0=False, 1=True

# 1-parent node -- 2 columns (parent has 2 states)
cpd_out = TabularCPD('Output', 2,
    [[0.9, 0.3], # Output=False: given P=0, given P=1
     [0.1, 0.7]], # Output=True
    evidence=['Child'], evidence_card=[2])

# 2-parent node -- 4 columns
# Col order: P1=0,P2=0 | P1=0,P2=1 | P1=1,P2=0 | P1=1,P2=1
cpd_child = TabularCPD('Child', 2,
    [[0.9, 0.7, 0.3, 0.1], # Child=False
     [0.1, 0.3, 0.7, 0.9]], # Child=True
    evidence=['P1', 'P2'], evidence_card=[2, 2])

model.add_cpds(cpd_p1, cpd_child, cpd_out)
assert model.check_model() # verify CPTs valid

infer = VariableElimination(model)
result = infer.query(['P1'], evidence={'Output':1})
print(result) # 0=No, 1=Yes in evidence
```

BN -- Burglary Alarm (Classic 5-node)

```
from pgmpy.models import DiscreteBayesianNetwork
from pgmpy.factors.discrete import TabularCPD
from pgmpy.inference import VariableElimination

model = DiscreteBayesianNetwork([
    ('burglary', 'alarm'), ('earthquake', 'alarm'),
    ('alarm', 'johncalls'), ('alarm', 'marycalls')
])

cpd_b = TabularCPD('burglary', 2, [[0.999],[0.001]])
cpd_e = TabularCPD('earthquake', 2, [[0.998],[0.002]])
cpd_a = TabularCPD('alarm', 2,
    [[0.999, 0.71, 0.06, 0.05],
     [0.001, 0.29, 0.94, 0.95]],
    evidence=['burglary', 'earthquake'], evidence_card=[2, 2])
cpd_j = TabularCPD('johncalls', 2,
    [[0.3, 0.9], [0.7, 0.1]], evidence=['alarm'], evidence_card=[2])
cpd_m = TabularCPD('marycalls', 2,
    [[0.2, 0.99], [0.8, 0.01]], evidence=['alarm'], evidence_card=[2])
model.add_cpds(cpd_b, cpd_e, cpd_a, cpd_j, cpd_m)
assert model.check_model()
infer = VariableElimination(model)
print(infer.query(['burglary'], evidence={'johncalls':1, 'marycalls':1}))
```

BN -- Disease Diagnosis

```
from pgmpy.models import DiscreteBayesianNetwork
from pgmpy.factors.discrete import TabularCPD
from pgmpy.inference import VariableElimination

model = DiscreteBayesianNetwork([
    ('Disease', 'Fever'), ('Disease', 'Cough'),
    ('Disease', 'Fatigue'), ('Disease', 'Chills')
])
cpd_d = TabularCPD('Disease', 2, [[0.3], [0.7]])
cpd_f = TabularCPD('Fever', 2, [[0.1, 0.5], [0.9, 0.5]],
                    evidence=['Disease'], evidence_card=[2])
cpd_c = TabularCPD('Cough', 2, [[0.2, 0.4], [0.8, 0.6]],
                    evidence=['Disease'], evidence_card=[2])
cpd_fa = TabularCPD('Fatigue', 2, [[0.3, 0.7], [0.7, 0.3]],
                    evidence=['Disease'], evidence_card=[2])
cpd_ch = TabularCPD('Chills', 2, [[0.4, 0.6], [0.6, 0.4]],
                    evidence=['Disease'], evidence_card=[2])
model.add_cpds(cpd_d, cpd_f, cpd_c, cpd_fa, cpd_ch)
assert model.check_model()
infer = VariableElimination(model)
print(infer.query(['Disease'], evidence={'Fever': 1, 'Cough': 1}))
```

Markov Model -- Weather Simulation

Markov Property: Future depends ONLY on current state, NOT history (memoryless). **Transition Matrix T:** $T[i][j] = P(\text{go to state } j \mid \text{in state } i)$. Each ROW sums to 1.

```
import numpy as np

states = ['Sunny', 'Cloudy', 'Rainy']
T = np.array([
    [0.6, 0.3, 0.1], # From Sunny
    [0.3, 0.4, 0.3], # From Cloudy
    [0.2, 0.3, 0.5] # From Rainy
])
# EACH ROW must sum to 1.0!

# Simulate one sequence
current = 0 # start = Sunny
seq = [states[current]]
for _ in range(10):
    current = np.random.choice(3, p=T[current])
    seq.append(states[current])
print(' -> '.join(seq))

# Monte Carlo: P(at least 3 rainy in 10 days | start=Sunny)
hits = 0
for _ in range(10000):
    cur = 0; cnt = 0
    for _ in range(10):
        cur = np.random.choice(3, p=T[cur])
        if cur == 2: cnt += 1
    if cnt >= 3: hits += 1
print(f'P(>=3 rainy) = {hits/10000:.4f}')
```

LAB 10 -- Supervised Learning

All Imports

```
import pandas as pd, numpy as np
import matplotlib.pyplot as plt, seaborn as sns
from sklearn.model_selection import train_test_split, KFold, cross_val_score
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.linear_model import LinearRegression, LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier, DecisionTreeRegressor
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import (accuracy_score, confusion_matrix,
                             classification_report, mean_squared_error, mean_absolute_error,
                             r2_score, roc_curve, roc_auc_score)
```

Data Preprocessing Cheatsheet

```
df = pd.read_csv('data.csv')
df.head(); df.info(); df.describe(); df.isnull().sum()

# Fill missing
df['num'].fillna(df['num'].mean(), inplace=True) # numeric
df['cat'].fillna(df['cat'].mode()[0], inplace=True) # categorical
df.dropna(inplace=True)

# Encoding
df['col'] = LabelEncoder().fit_transform(df['col']) # binary
df = pd.get_dummies(df, columns=['col'], drop_first=True) # multi-class

# Scaling (NEVER fit on test data!)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Remove outliers -- IQR
Q1,Q3 = df['col'].quantile([0.25,0.75]); IQR=Q3-Q1
df = df[(df['col']>=Q1-1.5*IQR)&(df['col']<=Q3+1.5*IQR)]

# Datetime features
df['Date'] = pd.to_datetime(df['Date'])
df['Month']=df['Date'].dt.month; df['Weekday']=df['Date'].dt.weekday
df['Hour']=df['Date'].dt.hour
```

Full Classification Template

```
def ml_classification():
    df = pd.read_csv('dataset.csv')
    for col in df.columns:
        if df[col].dtype=='object': df[col].fillna(df[col].mode()[0],inplace=True)
        else: df[col].fillna(df[col].mean(),inplace=True)
    df = df.drop_duplicates()

    target_col = 'target' # <-- CHANGE THIS

    for col in df.select_dtypes(include='object').columns:
        df = pd.get_dummies(df,columns=[col],drop_first=True)

    X=df.drop(columns=[target_col]); y=df[target_col]
    X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random_state=42)
    scaler=StandardScaler()
    X_train=scaler.fit_transform(X_train); X_test=scaler.transform(X_test)

    results={}
    for name,model in [
        ('LogisticReg', LogisticRegression(max_iter=1000)),
        ('KNN', KNeighborsClassifier(n_neighbors=5)),
```



```

        ('DecisionTree', DecisionTreeClassifier(random_state=42)),
        ('NaiveBayes', GaussianNB()),
    ]:
        model.fit(X_train,y_train); preds=model.predict(X_test)
        acc=accuracy_score(y_test,preds); results[name]=acc
        print(f'{name}: {acc:.4f}')
        print(classification_report(y_test,preds,zero_division=0))

    print(f'Best: {max(results,key=results.get)}')

ml_classification()

```

Full Regression Template

```

def ml_regression():
    df=pd.read_csv('dataset.csv')
    for col in df.columns:
        if df[col].dtype=='object': df[col].fillna(df[col].mode()[0],inplace=True)
        else: df[col].fillna(df[col].mean(),inplace=True)

    target_col='target' # <-- CHANGE THIS

    for col in df.select_dtypes(include='object').columns:
        df=pd.get_dummies(df,columns=[col],drop_first=True)

    X=df.drop(columns=[target_col]); y=df[target_col]
    X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random_state=42)
    scaler=StandardScaler()
    X_train=scaler.fit_transform(X_train); X_test=scaler.transform(X_test)

    for name,model in [
        ('LinearRegression', LinearRegression()),
        ('DecisionTree', DecisionTreeRegressor(random_state=42)),
    ]:
        model.fit(X_train,y_train); preds=model.predict(X_test)
        mse=mean_squared_error(y_test,preds)
        print(f'{name}: MSE={mse:.4f} RMSE={mse**0.5:.4f} '
              f'MAE={mean_absolute_error(y_test,preds):.4f} '
              f'R2={r2_score(y_test,preds):.4f}')

ml_regression()

```

K-Fold Cross Validation

```

from sklearn.model_selection import KFold, cross_val_score
import numpy as np

kf=KFold(n_splits=5,shuffle=True,random_state=42)
model=LogisticRegression(max_iter=1000)

# Quick (1 line)
scores=cross_val_score(model,X,y,cv=kf,scoring='accuracy')
print('Mean:', np.mean(scores).round(4))

# Manual (full control)
acc_list=[]
for tr,te in kf.split(X):
    X_tr,X_te=X.iloc[tr],X.iloc[te]; y_tr,y_te=y.iloc[tr],y.iloc[te]
    model.fit(X_tr,y_tr)
    acc_list.append(accuracy_score(y_te,model.predict(X_te)))
print('Manual Mean:', np.mean(acc_list).round(4))

```

Confusion Matrix + ROC Curve

```

# Confusion matrix
cm=confusion_matrix(y_test,y_pred)
sns.heatmap(cm,annot=True,fmt='d',cmap='Blues')
plt.xlabel('Predicted'); plt.ylabel('Actual'); plt.show()
print(classification_report(y_test,y_pred,zero_division=0))

```

```
# Metrics: Accuracy=(TP+TN)/Total Precision=TP/(TP+FP)
#          Recall=TP/(TP+FN)       F1=2*P*R/(P+R)
# TP=right positive TN=right negative
# FP=false alarm    FN=missed positive

# ROC Curve (binary only)
probs=model.predict_proba(X_test)[:,-1] # prob of class 1
fpr,tpr,_=roc_curve(y_test,probs)
auc=roc_auc_score(y_test,probs)
plt.plot(fpr,tpr,label=f'AUC={auc:.2f}')
plt.plot([0,1],[0,1],'--',color='gray')
plt.title('ROC Curve'); plt.legend(); plt.show()
# AUC=1.0 perfect | AUC=0.5 random | AUC>0.8 good
```

LAB 11 -- Unsupervised Learning (K-Means Clustering)

Concept

K-Means groups unlabeled data into K clusters. Steps: pick K centroids, assign points to nearest centroid, recalculate centroids as cluster means, repeat until stable. **WCSS (inertia)** = sum of squared distances to centroid -- lower is better.

Elbow Method: plot WCSS vs K, pick the sharp bend point as optimal K. **Always scale features before K-Means!**

Full K-Means Template

```
import pandas as pd, matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans

def ml_unsupervised():
    df=pd.read_csv('dataset.csv')
    for col in df.columns:
        if df[col].dtype=='object': df[col].fillna(df[col].mode()[0],inplace=True)
        else: df[col].fillna(df[col].mean(),inplace=True)
    df=df.drop_duplicates()
    for col in df.select_dtypes(include='object').columns:
        df=pd.get_dummies(df,columns=[col],drop_first=True)

    X=df[['col1','col2']].values    # <-- CHANGE COLUMNS

    scaler=StandardScaler()
    X_scaled=scaler.fit_transform(X)

    # Elbow method
    wcss=[]
    for k in range(1,11):
        km=KMeans(n_clusters=k,init='k-means++',random_state=42,n_init=10)
        km.fit(X_scaled); wcss.append(km.inertia_)
    plt.plot(range(1,11),wcss,marker='o')
    plt.title('Elbow Method'); plt.xlabel('K'); plt.ylabel('WCSS'); plt.show()

    K=3    # <-- CHANGE after looking at elbow
    kmeans=KMeans(n_clusters=K,init='k-means++',random_state=42,n_init=10)
    labels=kmeans.fit_predict(X_scaled)
    df['Cluster']=labels

    colors=['blue','green','red','black','purple']
    for k in range(K):
        mask=labels==k
        plt.scatter(X[mask,0],X[mask,1],s=80,c=colors[k],label=f'Cluster {k+1}')
    plt.scatter(kmeans.cluster_centers_[0,0],kmeans.cluster_centers_[0,1],
                s=300,c='yellow',marker='*',label='Centroids')
    plt.legend(); plt.title('K-Means'); plt.show()
    print(df['Cluster'].value_counts())

ml_unsupervised()
```

K-Means with Train/Test Split + PCA Visualization

```
import pandas as pd, matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.model_selection import train_test_split

df=pd.read_csv('dataset.csv')

# Aggregate to customer level (for retail datasets)
cust=df.groupby('CustomerID').agg(
    total_spend=('TotalPrice','sum'),
    freq=('InvoiceNo','nunique'),
    qty=('Quantity','sum')
```

```

).reset_index()

X=cust[['total_spend','freq','qty']]
X_train,X_test=train_test_split(X,test_size=0.2,random_state=42)

scaler=StandardScaler()
Xtr=scaler.fit_transform(X_train) # fit on train only!
Xte=scaler.transform(X_test)

# Elbow
wcss=[KMeans(n_clusters=k,random_state=42).fit(Xtr).inertia_ for k in range(1,11)]
plt.plot(range(1,11),wcss,marker='o'); plt.title('Elbow'); plt.show()

K=3 # <-- pick from elbow
kmeans=KMeans(n_clusters=K,init='k-means++',random_state=42)
kmeans.fit(Xtr)
train_labels=kmeans.labels_ # labels for training data
test_labels=kmeans.predict(Xte) # predict labels for test data

# PCA for 2D visualization
pca=PCA(n_components=2)
tr_pca=pca.fit_transform(Xtr) # fit+transform on train
te_pca=pca.transform(Xte) # only transform on test

colors=['red','blue','green','purple','orange']
fig,axes=plt.subplots(1,2,figsize=(12,4))
for c in range(K):
    axes[0].scatter(tr_pca[train_labels==c,0],tr_pca[train_labels==c,1],
                    c=colors[c],label=f'C{c}',alpha=0.7)
    axes[1].scatter(te_pca[test_labels==c,0], te_pca[test_labels==c,1],
                    c=colors[c],label=f'C{c}',alpha=0.7)
axes[0].set_title('Train Clusters (PCA)'); axes[1].set_title('Test Clusters (PCA)')
for ax in axes: ax.legend()
plt.tight_layout(); plt.show()

```

K-Means Key Params & Attributes

```

KMeans(
    n_clusters=5, # K -- number of clusters
    init='k-means++', # smart initialization (better than random)
    random_state=42, # reproducible
    n_init=10 # run 10 times, pick best WCSS
)

# After fitting:
kmeans.inertia_ # WCSS (lower = tighter clusters)
kmeans.cluster_centers_ # centroid coords [K x features]
kmeans.labels_ # label for each training point

kmeans.fit(X) # train
kmeans.fit_predict(X) # train + return labels
kmeans.predict(X_new) # predict for new data

# PCA:
from sklearn.decomposition import PCA
pca=PCA(n_components=2)
X_2d=pca.fit_transform(X_scaled) # fit+transform on train
X_test_2d=pca.transform(X_test) # only transform on test

```

Quick Reference -- All Key Snippets

CSP -- Key Functions

```
# Variable types
x = model.NewIntVar(min, max, 'name')      # integer variable
b = model.NewBoolVar('name')              # boolean 0 or 1
interval = model.NewFixedSizeIntervalVar(start, size, 'name')

# Constraint types
model.AddAllDifferent([x,y,z])             # all must differ
model.Add(x < y)                           # ordering
model.Add(x != 3)                         # not equal
model.AddAbsEquality(abs_var, diff_var)    # abs value
model.AddForbiddenAssignments([x],[ (3,)] ) # x cannot be 3
model.AddAllowedAssignments([x],[ (2,),(4,)] ) # x can only be 2 or 4
model.AddBoolOr([b1,b2])                  # at least one true
model.Add(x==3).OnlyEnforceIf(b)           # conditional
model.Add(x!=3).OnlyEnforceIf(b.Not())

# Solve
status=solver.Solve(model)
if status in (cp_model.FEASIBLE,cp_model.OPTIMAL):
    print(solver.Value(x)) # get assigned value
# Statuses: OPTIMAL FEASIBLE INFEASIBLE UNKNOWN
```

EDA -- Full Retail Dataset Checklist

```
df=pd.read_csv('retail.csv')
df.shape; df.dtypes; df.describe(); df.isnull().sum()
df=df.dropna(subset=['CustomerID'])        # drop if ID missing
df['Description']=df['Description'].fillna('Unknown')
df=df[df['Quantity']>0]                   # remove returns
df=df[df['UnitPrice']>0]                  # remove invalid
df['InvoiceDate']=pd.to_datetime(df['InvoiceDate'])
df['Month']=df['InvoiceDate'].dt.month
df['Weekday']=df['InvoiceDate'].dt.weekday
df['TotalPrice']=df['Quantity']*df['UnitPrice']

# Outliers -- IQR
for col in ['Quantity','UnitPrice']:
    Q1,Q3=df[col].quantile([0.25,0.75]); IQR=Q3-Q1
    df=df[(df[col]>=Q1-1.5*IQR)&(df[col]<=Q3+1.5*IQR)]

# Aggregate
cust=df.groupby('CustomerID').agg(
    total_spend=('TotalPrice','sum'),
    freq=('InvoiceNo','nunique'),
    qty=('Quantity','sum')
).reset_index()
```

Minimax vs Alpha-Beta at a Glance

```
# MINIMAX: O(b^d) -- checks ALL nodes
def minimax(node,is_max):
    if isinstance(node,int): return node
    vals=[minimax(c,not is_max) for c in node]
    return max(vals) if is_max else min(vals)

# ALPHA-BETA: O(b^(d/2)) -- prunes useless branches
# Call: alphabeta(root, float('-inf'), float('inf'), True)
# At MAX node: alpha=max(alpha,val); if alpha>=beta: break
# At MIN node: beta=min(beta,val); if alpha>=beta: break

# Markov: future depends ONLY on present
# T[i][j] = P(go to j | currently in i) -- rows sum to 1
# next=np.random.choice(states,p=T[current])
```

```
# BN: DAG + CPTs. Each CPT COLUMN sums to 1.
# Roots: TabularCPD('X',2,[[p0],[p1]])
# 1 parent: TabularCPD('X',2,[[a,b],[c,d]],
#               evidence=['P'],evidence_card=[2])
# 2 parents: 4 columns (P1=0,P2=0|P1=0,P2=1|P1=1,P2=0|P1=1,P2=1)
```

Metrics Summary Table

Classification: Accuracy | F1 | Precision | Recall | ROC-AUC | Confusion Matrix

Regression: MSE (mean sq error) | RMSE (sqrt MSE) | MAE (mean abs error) | R2 (closer to 1 = better)

Clustering: WCSS/Inertia (lower = tighter) | Elbow Method for optimal K

✓ Train-test split: fit scaler/encoder on TRAIN only, then transform BOTH. Never fit on test data -- that is data leakage!